SOFTWARE PRODUCTION ENGINEERING

Mini Project

End-to-End CI/CD Pipeline for a Calculator REST API

Submitted By

IMT2023057 – K. Jitin Venkata Sai

GitHub Link

`https://github.com/jitin1975/calculator-devops`

Docker Hub Link

`https://hub.docker.com/r/jitin1975/calculator-devops`

Course: Software Production Engineering (CSE 816)

Faculty: Prof. B. Thangaraju

Date: 22 October 2026

1. Objective

The goal of this mini-project is to build a small but production-style Calculator REST API in Java (Spring Boot) and wrap it in a fully automated DevOps pipeline using Git/GitHub, Jenkins, Docker, and Docker Compose. The application itself is kept deliberately simple; the real work is in the automation around it. Once set up, a single push to GitHub builds, tests, packages, containerizes, publishes, and redeploys the service with no manual steps.

2. Pipeline Architecture

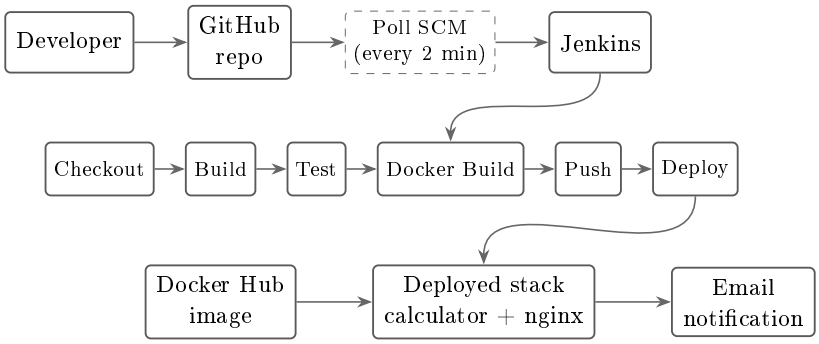


Figure: flow of the automated pipeline, from a push to the redeployed stack.

The declarative pipeline is defined in the `Jenkinsfile` and runs six stages. It is triggered automatically: Jenkins is configured with Poll SCM (it checks the repository every two minutes), so a push to GitHub starts a build on its own, as shown in Figure 2. The image is tagged with the Jenkins build number and pushed to Docker Hub using credentials stored in the Jenkins credentials store; the Deploy stage then brings the stack up and verifies its health before the build is marked successful. Build status is emailed on both success and failure.

3. The Application

The service is built with Spring Boot (Spring Web and Actuator) using Maven on Java 21, and produces an executable JAR. There are four GET endpoints under `/api/v1` — `add`, `subtract`, `multiply` and `divide` — each taking two numeric parameters and returning a JSON response, for example:

```
{"operation":"add","a":2,"b":3,"result":5}
```

A controller-advice handler returns a structured HTTP 400 response for missing or non-numeric parameters and for division by zero. Actuator exposes `/actuator/health`, which the Compose health check relies on. Testing uses JUnit 5, with a unit test for every operation (including the divide-by-zero case) and a Spring MVC `MockMvc` integration test that hits the real endpoints. All ten tests pass, and the JUnit results are published by Jenkins.

4. Pipeline Stages

Stage	What it does
Checkout	Pulls the triggering commit from GitHub.
Build	Runs `mvn clean package -DskipTests` to produce the JAR.
Test	Runs `mvn test` and publishes the JUnit results; the build fails if any test fails.
Docker Build	Builds the image with the multi-stage Dockerfile, tagged with the build number and `latest`.
Push	Logs in with the stored Docker Hub credential and pushes both tags.
Deploy	Runs `docker compose up -d`, then polls `/actuator/health` until the service reports UP.

5. Containerization and Deployment

The Dockerfile uses a multi-stage build: a Maven/JDK stage compiles the JAR with a cached dependency layer, and a slim `eclipse-temurin:21-jre-alpine` stage runs it as a non-root user with an explicit exposed port. A `.dockerignore` keeps the build context small. The `docker-compose.yml` file brings up two components — the calculator and an Nginx reverse proxy in front of it — on a user-defined bridge network, with a health check on `/actuator/health`, a restart policy, and environment variables. Only the proxy is published to the host (on port 8090); the API is reached through it. Because the container names are fixed, re-running the pipeline simply recreates the same stack, which keeps the deployment idempotent as required.

6. Challenges Faced

- **JDK version mismatch.** Spring Boot 3 needs Java 17 or newer, but my WSL setup defaulted to Java 8, so the first build failed with a "wrong class file version" error. Installing JDK 21 and pointing `JAVA_HOME` at it fixed the build.
- **Integration test silently skipped.** I first named the MockMvc test with an `IT` suffix, which Surefire ignores (that convention belongs to Failsafe), so `mvn test` quietly ran only five of the ten tests until I renamed it to end in `Test`.
- **Docker socket permission denied.** Jenkins runs as its own service user, which was not in the `docker` group, so the Docker Build stage failed with a permission-denied error on the Docker socket. Adding the `jenkins` user to the `docker` group and restarting the service resolved it.
- **Host port 80 conflict.** Port 80 was already in use on my machine, so the Nginx proxy could not bind to it. I remapped the proxy to host port 8090.
- **Compose v1 versus v2.** My WSL has the standalone `docker-compose` (v1), while the Jenkinsfile originally used the `docker compose` (v2) syntax. I updated the Deploy stage to detect whichever version is available before running it.
- **Health check on the wrong port.** The Compose file only publishes the proxy (8090); the calculator's own port 8080 stays internal. My deploy health check first queried port 8080 and timed out even though the app was running — pointing it at the proxy on 8090 (which Nginx forwards to the API) fixed it.

7. Results

With everything in place, a single push drives the whole pipeline to a successful run and an automatic redeploy. The automatic trigger was verified by pushing a commit and letting Jenkins start the build on its own (Figure 2). The figures below show the full green run, the auto-triggered build, the status email, and the running containers with sample API responses.

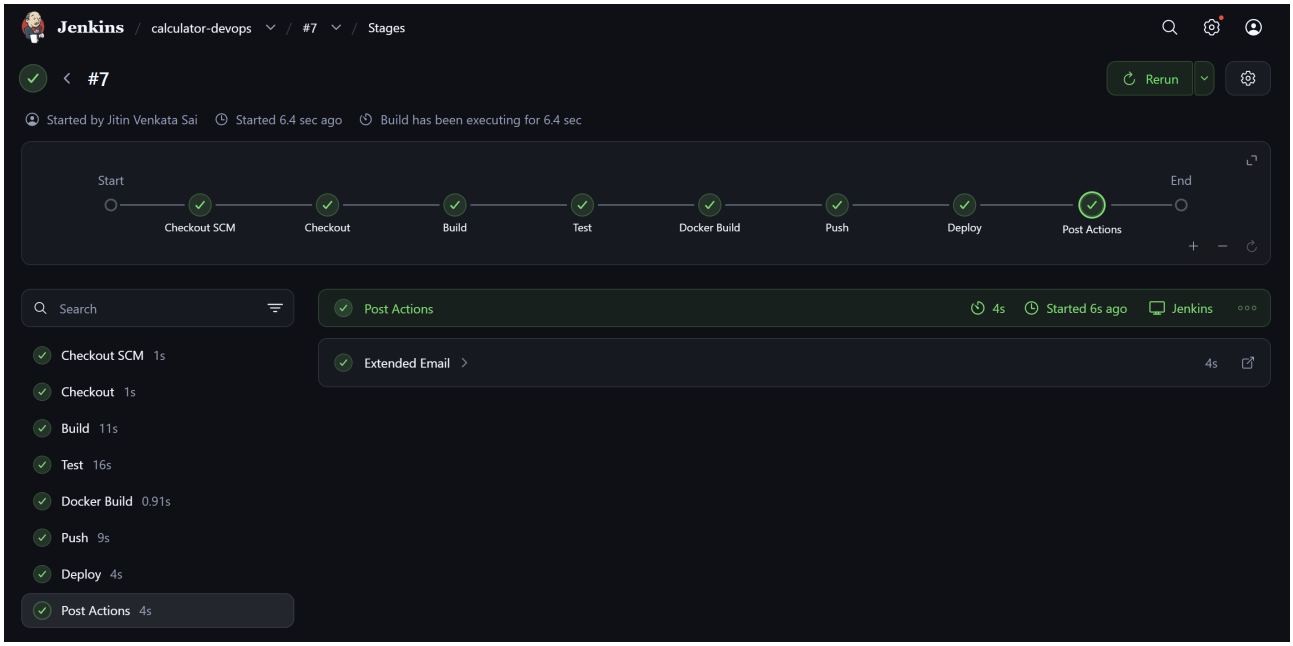


Figure 1: A full successful Jenkins pipeline run (build #7): all six stages, plus the post-build email.



Figure 2: Build #8 was started automatically by a source-code change (Poll SCM), with no manual start.

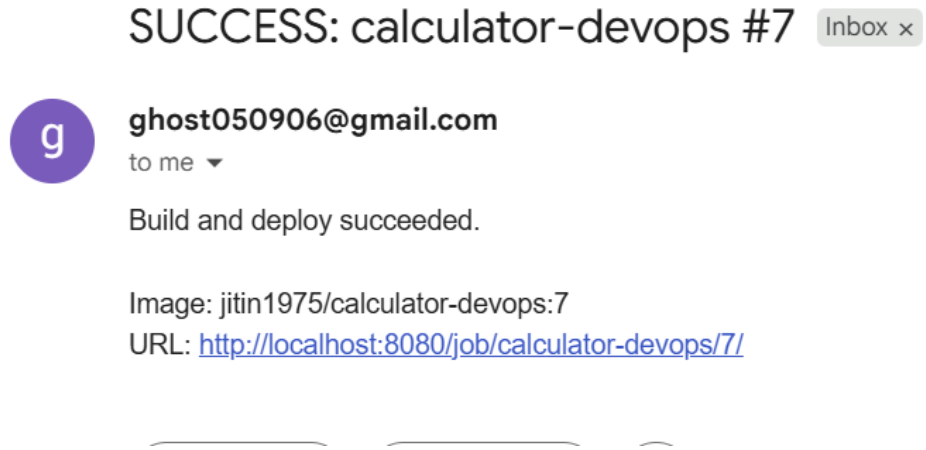


Figure 3: The build-status email sent by the pipeline on a successful run.

```
jitin@Jitin:~/calculator-devops$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED          STATUS                    PORTS
          NAMES
f07261730b24   nginx:alpine                         "/docker-entrypoint.…"   11 minutes ago   Up 11 minutes             0.0.0.0:8090->80/tcp, [::]:8
090->80/tcp   calculator-proxy
c58aae17d0e4   jitin1975/calculator-devops:latest   "java -jar app.jar"      51 minutes ago   Up 51 minutes (healthy)   8080/tcp
          calculator
jitin@Jitin:~/calculator-devops$  curl "http://localhost:8090/api/v1/add?a=2&b=3"; echo
{"operation":"add","a":2.0,"b":3.0,"result":5.0}
jitin@Jitin:~/calculator-devops$    curl "http://localhost:8090/api/v1/divide?a=6&b=0"; echo
{"timestamp":"2026-10-22T11:11:34.142810811Z","status":400,"error":"division by zero is not allowed","path":"/api/v1/divide"}
jitin@Jitin:~/calculator-devops$    curl "http://localhost:8090/actuator/health"; echo
{"status":"UP","components":{"diskSpace":{"status":"UP","details":{"total":1081101176832,"free":999854915584,"threshold":10485760,"path":"/app/.","e
xists":true}},"ping":{"status":"UP"}}}
jitin@Jitin:~/calculator-devops$
```

Figure 4: `docker ps` showing the running calculator (healthy) and proxy containers, with sample API responses: a successful addition, the divide-by-zero HTTP 400, and the health probe.